Chiffrement AES et chiffrement par blocs

30 novembre 2021

Table des matières

1 Chiffrement AES **2**
1.1 Préambule . . . 2
1.1.1 Une distinction importante . . . 2
1.1.2 Vocabulaire . . . 2
1.2 Algorithme . . . 3
1.2.1 Vue d'ensemble . . . 3
1.2.2 Préparatifs . . . 3
1.2.2.1 Découpage des données . . . 3
1.2.2.2 Expansion de clé . . . 4
1.2.3 Détail des étapes . . . 5
1.2.3.1 Substitution par Octet . . . 5
1.2.3.2 Décalage par ligne . . . 5
1.2.3.3 Mélange des colonnes . . . 5
1.2.3.4 Addition de la sous-clé . . . 6
1.3 Déchiffrement . . . 7
1.3.1 Principe du déchiffrement . . . 7
1.3.2 Détail des opérations inverses . . . 7
1.3.2.1 Inverse de la substitution par octet . . . 7
1.3.2.2 Inverse du décalage par lignes . . . 7
1.3.2.3 Inverse du mélange par colonne . . . 8
1.3.2.4 Inverse de l'addition de la sous-clé . . . 9
1.4 Implémentation . . . 9
1.4.1 Génération de la clé de chiffrement . . . 9

2 Mode d'opération **9**
2.1 Dictionnaire de codes : « Electronic codebook » (ECB) . . . 9
2.2 Enchaînement des blocs : « Cipher Block Chaining » (CBC) . 10
2.3 Chiffrement basé sur un compteur : « CounTeR » (CTR) . . 11

3 Attaques **12**
3.1 Attaque par manipulation de bits 12
3.2 Injection de blocs 13
3.3 Oracle de remplissage 14
3.3.1 Principe : ce qu'il faut comprendre 14
3.3.2 Explication détaillée 15

A Fonction de substitution **16**

1 Chiffrement AES

1.1 Préambule

Le chiffrement AES (Advanced Encryption Standard) est un algorithme de chiffrement symétrique, c'est-à-dire que les clés de chiffrement et de déchiffrement sont les mêmes. Il a été créé par les chercheurs belges Joan Daemen et Vincent Rijmen en 2000 lors d'un concours pour déterminer un nouveau standard de cryptographie. Cet algorithme avait trois objectifs, celui de résister aux attaques connues, d'être rapide à exécuter et enfin d'être simple à implémenter. Cet algorithme est encore l'un des plus utilisés de nos jours. Avant d'être sélectionné lors du concours, il avait pour nom *Rijndael*.

1.1.1 Une distinction importante

Avec le chiffrement AES, il est important de différencier le *mot de passe*, et la *clé de chiffrement*. Le mot de passe est la chaîne de caractères que vous envoyez dans le programme de déchiffrement, et la clé de déchiffrement est un suite d'octets utilisée par l'algorithme pour effectuer les opérations sur les données chiffrées. Bien évidemment, en pratique vous ne connaissez que votre mot de passe. Le programme va en fait transformer le mot de passe en clé de chiffrement, et cette clé a une taille spécifique. Dans le cas de l'AES, elle dépend de la spécification, elle peut faire *exactement* soit 128, soit 192 ou soit 256 bits.

1.1.2 Vocabulaire

— Mot (*word*) : un *mot* est un regroupement de quatre octets, donc correspond à un objet de 32 bits.
— Fonction de dérivation de clé (*key derivation function*, KDF) : fonction permettant de générer une clé de chiffrement à partir d'un mot de passe par exemple.
— Table-S (S-Box) : Table de substitution permettant de casser la linéarité d'un algorithme de chiffrement de manière à empêcher des analyses.

— Oracle : procédure capable d'analyser le comportement observé du programme exécuté, et de décider si oui ou non une défaillance est présente.
— Nonce : nombre arbitraire utilisé une seule fois dans une communication cryptographique, il peut être aléatoire ou pseudo-aléatoire. Originaire de « *þe naness* », une expression anglaise du XIIIe siècle signifiant « pour un but particulier ».

1.2 Algorithme

1.2.1 Vue d'ensemble

Pour chaque étape, les données utilisées sont celles obtenues à l'étape précédente.

Voyons le déroulé général de l'algorithme :

1. Expansion de clé
2. Ajout de la sous-clé
3. 9, 11 ou 13 tours :
   (a) Substitution par octet
   (b) Décalage par ligne
   (c) Mélange des colonnes
   (d) Addition de la sous-clé numéro i
4. Dernier tour
   (a) Substitution par octet
   (b) Décalage par ligne
   (c) Addition de la sous clé

Le nombre de tours de chiffrement dépend directement de la taille de la clé de chiffrement. Pour une clé de 128 bits, il y a 9 tours en tout (en comptant le premier tours), pour 192 bits il y en a 11, et pour 256 bits il y en a 13. L'étape 3 est donc répétée respectivement 9, 11 et 13 fois.

Au dernier tour il n'y a pas de mélange des colonnes !!!

1.2.2 Préparatifs

1.2.2.1 Découpage des données On découpe les données à chiffrer en blocs de 128 bits, que l'on découpe à leur tour en blocs d'un octet que l'on dispose dans une matrice carrée de taille 4.

$$\begin{pmatrix} X_1 & X_2 & X_3 & X_4 \\ X_5 & X_6 & X_7 & X_8 \\ X_9 & X_{10} & X_{11} & X_{12} \\ X_{13} & X_{14} & X_{15} & X_{16} \end{pmatrix} \tag{1}$$

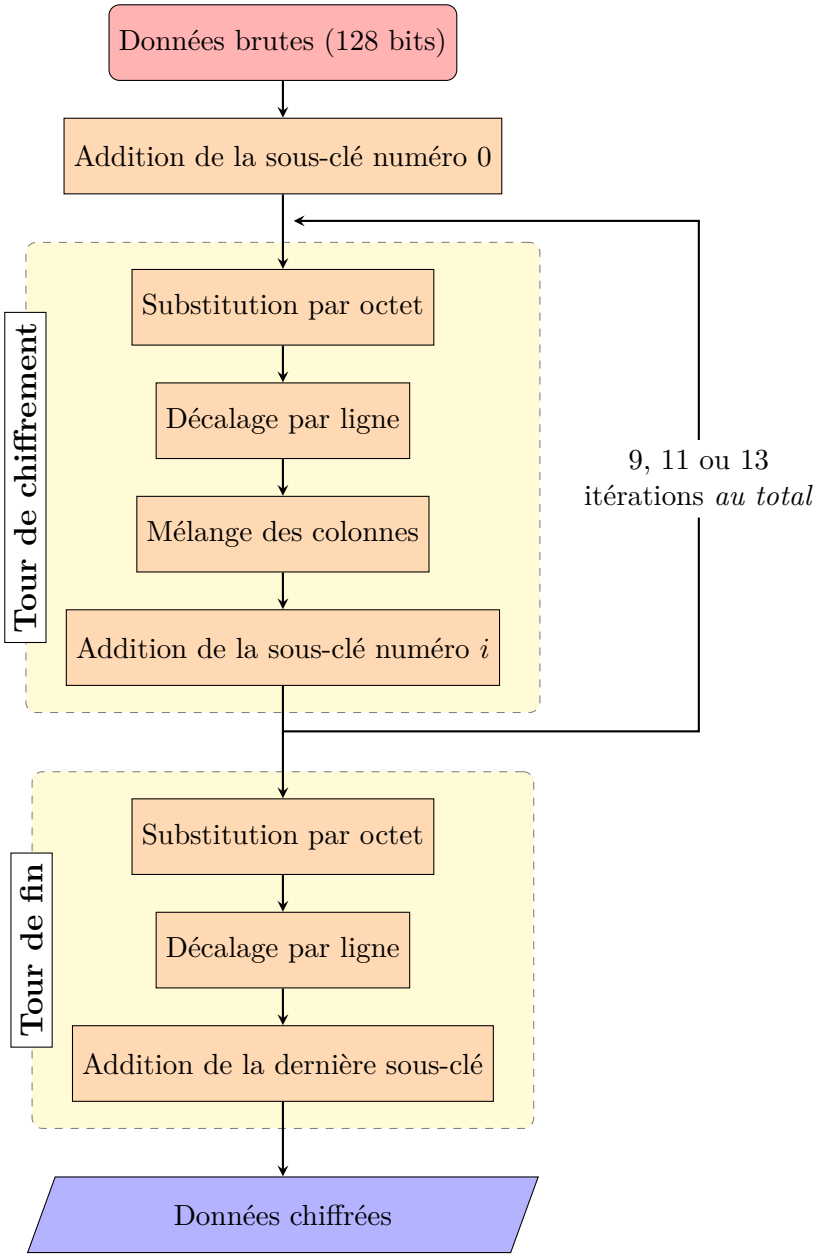

FIGURE 1 – Principe général de l'algorithme

Les éléments $X_i, i \in [\![3;4]\!]$ représentent donc chacun un octet de données.

1.2.2.2 Expansion de clé Pour complexifier le chiffrement, l'étape d'ajout de la sous-clé utilise une sous-clé différente à chaque tour. Pour cela, on effectue une *expansion de clé*, et cette étape dépend de la taille de la clé initiale, donc 128, 196 ou 256 bits.

La procédure détaillée est relativement complexe et assez peu intéressante, il faut juste retenir que l'on obtient une suite d'éléments (k_n) de taille dépendant de la taille de la clé, et chaque élément a pour taille 16×8 bits.

Taille de la clé (en bits)	128	192	256
Expansion de clé (en mots, 32 bits)	44	52	60
Expansion de clé (16×8 bits)	11	13	15
Nombre de tours de chiffrement	9	11	13

TABLE 1 – Paramètres

1.2.3 Détail des étapes

1.2.3.1 Substitution par Octet On définit une fonction de substitution S selon la table de substitution de Rijndael définie en annexe. On applique cette fonction S sur les éléments de la matrice précédente.

$$\begin{pmatrix} X_1 & X_2 & X_3 & X_4 \\ X_5 & X_6 & X_7 & X_8 \\ X_9 & X_{10} & X_{11} & X_{12} \\ X_{13} & X_{14} & X_{15} & X_{16} \end{pmatrix} \longrightarrow \begin{pmatrix} S(X_1) & S(X_2) & S(X_3) & S(X_4) \\ S(X_5) & S(X_6) & S(X_7) & S(X_8) \\ S(X_9) & S(X_{10}) & S(X_{11}) & S(X_{12}) \\ S(X_{13}) & S(X_{14}) & S(X_{15}) & S(X_{16}) \end{pmatrix} \quad (2)$$

1.2.3.2 Décalage par ligne On effectue ensuite une opération sur les lignes. Elle consiste à décaler de i cases vers la gauche les cases du tableau, i étant le numéro de la ligne.

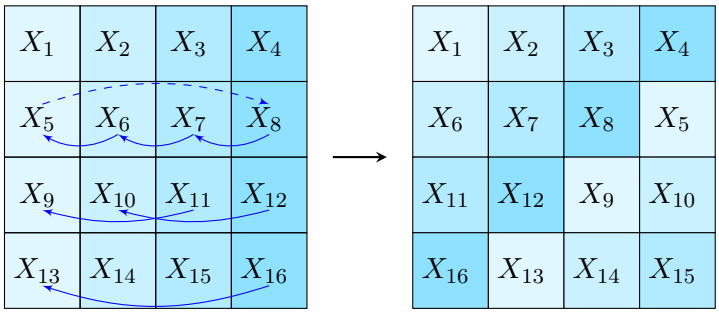

FIGURE 2 – Mélange par colonnes

1.2.3.3 Mélange des colonnes On fait ensuite un transformation linéaire sur chaque colonne. Cette opération est sûrement la plus délicate à comprendre. Prenons par exemple la première colonne $\begin{pmatrix} X_1 \\ X_5 \\ X_9 \\ X_{13} \end{pmatrix}$ obtenue lors

de l'étape précédente.

$$\begin{pmatrix} Y_1 \\ Y_5 \\ Y_9 \\ Y_{13} \end{pmatrix} = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \times \begin{pmatrix} X_1 \\ X_5 \\ X_9 \\ X_{13} \end{pmatrix} \tag{3}$$

La matrice $\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$ n'a pas été choisie arbitrairement, mais la justification est relativement complexe, donc elle ne sera pas abordée plus en profondeur ici.

L'opération s'effectue sur $GF(2^8)$ (GF représentant un groupe de Galois, ou corps fini).

Passons également les détails pour aller directement aux règles de calcul. On note que tous les éléments sont des octets, les éléments des colonnes ainsi que les éléments de la matrice carrée utilisée pour la multiplication, et il faut également que les sorties soient des octets. Il faut donc définir des règles de calcul compatibles avec ces conditions.

L'addition est assez simple à mettre en œuvre, elle consiste en une simple opération OU EXCLUSIF, soit XOR. Par exemple $00101011_2 \oplus 10011010_2 = 10110001_2$

Pour la multiplication, c'est un peu plus complexe. On commence par représenter les octets par des polynômes de degré au plus 7. Par exemple, 0x01h a pour représentation 0000 0001 en binaire, et sa représentation polynomiale est 1. 0x02h est 0000 0010 en binaire, et X en polynomial. 0x03h est 0000 0011 en binaire, donc $X + 1$ en polynomial et ainsi de suite.

Une fois cette représentation faite, on passe à la multiplication proprement dite. On passe une fois de plus les détails, mais dans un groupe de Gallois, on utilise un polynôme irréductible afin de construire le corps fini. Dans le cas de l'AES pour le corps fini $GF(2^8)$, on utilise le polynôme $P = X^8 + X^4 + X^3 + X + 1$. La multiplication de deux polynômes dans ce groupe se fait donc de la manière habituelle, mais modulo P. Si on pose $A = \sum_{i=0}^{7} a_i X^i$ et $B = \sum_{i=0}^{7} b_i X^i$ deux polynômes, on a alors

$$A * B = \sum_{i=0}^{7} a_i X^i \times \sum_{i=0}^{7} b_i X^i \mod P$$

On peut désormais effectuer l'opération de la multiplication de la colonne par la matrice.

1.2.3.4 Addition de la sous-clé Cette opération est assez directe. On reprend la suite générée (k_n) générée lors de l'expansion de clé, on orga-

nise l'élément i dans une matrice comme pour les données, et on effectue l'opération XOR entre la matrice de données et la matrice de la sous clé.

$$\begin{aligned}
&\begin{pmatrix} X_1 & X_2 & X_3 & X_4 \\ X_5 & X_6 & X_7 & X_8 \\ X_9 & X_{10} & X_{11} & X_{12} \\ X_{13} & X_{14} & X_{15} & X_{16} \end{pmatrix} \oplus \begin{pmatrix} K_{i,1} & K_{i,2} & K_{i,3} & K_{i,4} \\ K_{i,5} & K_{i,6} & K_{i,7} & K_{i,8} \\ K_{i,9} & K_{i,10} & K_{i,11} & K_{i,12} \\ K_{i,13} & K_{i,14} & K_{i,15} & K_{i,16} \end{pmatrix} \\
&= \begin{pmatrix} X_1 \oplus K_{i,1} & X_2 \oplus K_{i,2} & X_3 \oplus K_{i,3} & X_4 \oplus K_{i,4} \\ X_5 \oplus K_{i,5} & X_6 \oplus K_{i,6} & X_7 \oplus K_{i,7} & X_8 \oplus K_{i,8} \\ X_9 \oplus K_{i,9} & X_{10} \oplus K_{i,10} & X_{11} \oplus K_{i,11} & X_{12} \oplus K_{i,12} \\ X_{13} \oplus K_{i,13} & X_{14} \oplus K_{i,14} & X_{15} \oplus K_{i,15} & X_{16} \oplus K_{i,16} \end{pmatrix}
\end{aligned} \tag{4}$$

1.3 Déchiffrement

1.3.1 Principe du déchiffrement

Quand on a compris le protocole de chiffrement, la procédure de déchiffrement est assez simple. Elle consiste tout simplement à appliquer les fonctions inverses de chiffrement dans l'ordre inverse. En effet, toutes les opérations effectuées précédemment sont individuellement inversibles.

La procédure est donc la suivante :

1. Expansion de clé
2. Ajout de la sous-clé
3. Inverse du décalage par ligne
4. Inverse de la substitution par octet
5. 9, 11 ou 13 tours :
   (a) Addition de la sous-clé numéro i
   (b) Inverse du mélange des colonnes
   (c) Inverse du décalage par ligne
   (d) Inverse de la substitution par octet
6. Dernier tour
   (a) Addition de la sous clé

1.3.2 Détail des opérations inverses

1.3.2.1 Inverse de la substitution par octet Même principe que la substitution normale, on utilise juste l'inverse de la fonction S à la place, voir annexe pour la fonction inverse.

1.3.2.2 Inverse du décalage par lignes Même principe que la fonction directe, mais on compense le décalage précédent. Donc on ne décale pas la première ligne, on déplace d'un cran vers la droite la deuxième...

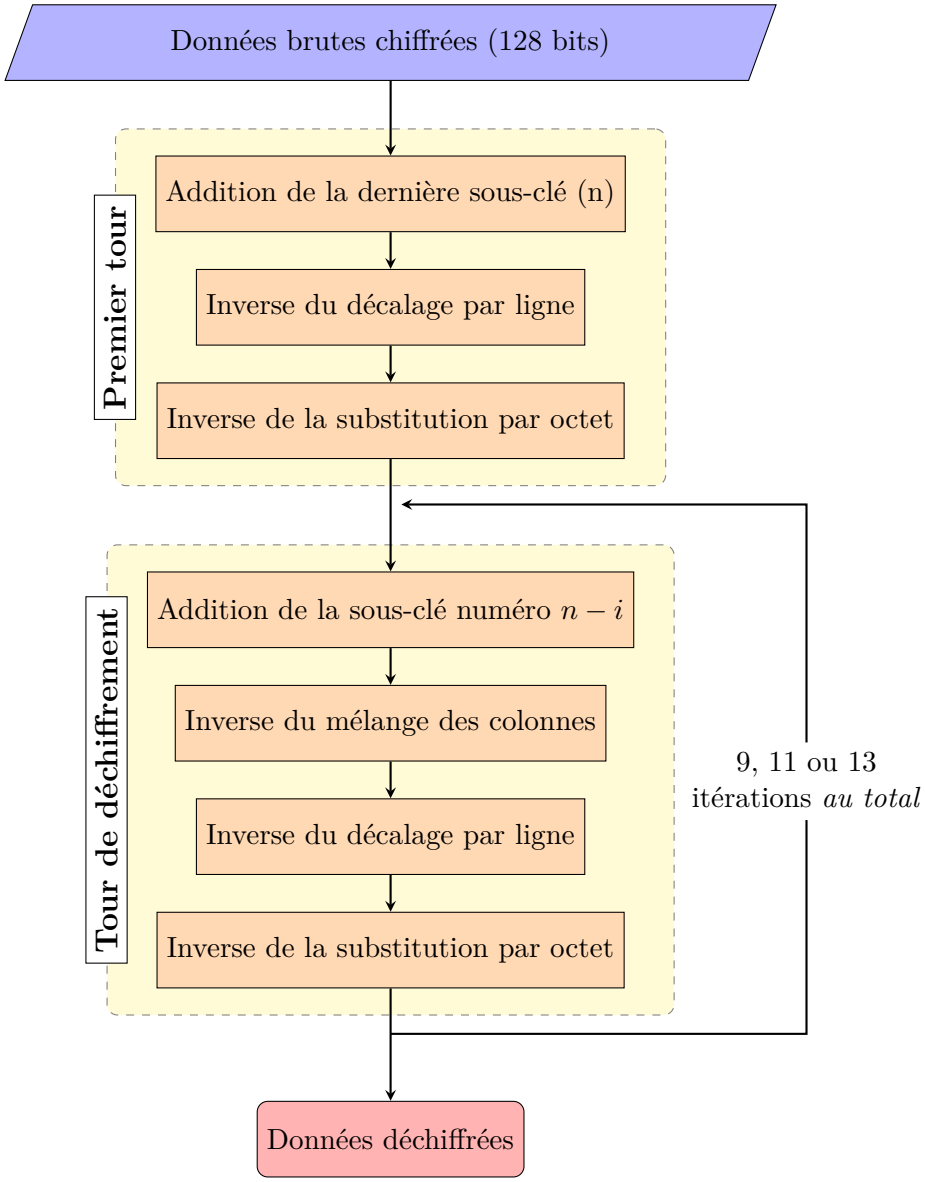

FIGURE 3 – Principe général de l'algorithme

1.3.2.3 Inverse du mélange par colonne La procédure est identique au mélange par colonne du chiffrement, au détail près que la matrice utilisée est la suivante : $\begin{pmatrix} 14 & 11 & 13 & 9 \\ 9 & 14 & 11 & 13 \\ 13 & 9 & 14 & 11 \\ 11 & 13 & 9 & 14 \end{pmatrix}$

Les opérations suivent les mêmes règles que vues précédemment.

1.3.2.4 Inverse de l'addition de la sous-clé L'addition de la sous clé se faisant avec l'opération OU EXCLUSIF, il est simple d'inverse cette fonction. En effet, d'après les propriétés du XOR, $(A \oplus B) \oplus B = A$. Il suffit donc de répéter l'opération d'addition de la sous clé à l'identique en utilisant les données chiffrées à la place.

1.4 Implémentation

1.4.1 Génération de la clé de chiffrement

Comme évoqué en introduction, il faut faire la différence entre le mot de passe et la clé de chiffrement. Mais comment générer une clé de chiffrement à partir d'un mot de passe ? Pour cela, on utilise une *fonction de dérivation de clé*. Il s'agit d'une fonction qui va transformer le mot de passe en clé de chiffrement en utilisant un procédé *pseudo-aléatoire*. Pour faire simple, c'est une fonction déterministe dont la sortie n'est pas efficacement distinguable des sorties d'une fonction aléatoire. Cela correspond au fonctionnement des fonctions de hachages telles que SHA-1, SHA-256, MD5... On peut citer PBKDF2 pour ce procédé. Par exemple, le logiciel libre bitwarden utilise cet algorithme pour générer la clé de chiffrement.

Il est à noter que ces fonctions de génération opèrent à un niveau binaire, par conséquent l'encodage de caractères du mot de passe peut faire une différence. Le sous-système *Crypsetup* du noyau Linux, standard *de-facto* du chiffrement pour les système basés sur Linux, recommande justement de n'utiliser que des caractères de la table ASCII pour éviter les problèmes. Par ailleurs, *Cryptsetup* utilise la fonction de génération de clé *Argon2*.

Ces fonctions appliquent une procédure plus ou moins complexe pour générer la clé, certaines fonctions sont conçues spécifiquement pour être lentes, afin de complexifier la tâche d'un éventuel pirate.

2 Mode d'opération

L'AES est ce qu'on appelle un chiffrement par blocs : c'est à dire que l'on découpe les données à chiffrer en morceaux d'une certaine taille, puis on applique des opérations sur ces blocs. Cependant découper les données en blocs puis les chiffrer indépendamment n'est pas la seule méthode utilisée.

2.1 Dictionnaire de codes : « Electronic codebook » (ECB)

L'ECB est la méthode la plus simple et la plus directe d'utilisation de l'AES. Elle consiste tout simplement à découper les données en blocs de 128 bits, puis de les chiffrer indépendamment par AES, avant de les concaténer pour former le fichier chiffré.

Cependant, cette méthode est malheureusement problématique. En effet, si il y a deux blocs identiques dans les données initiales, alors il y aura deux blocs identiques dans les données chiffrées.

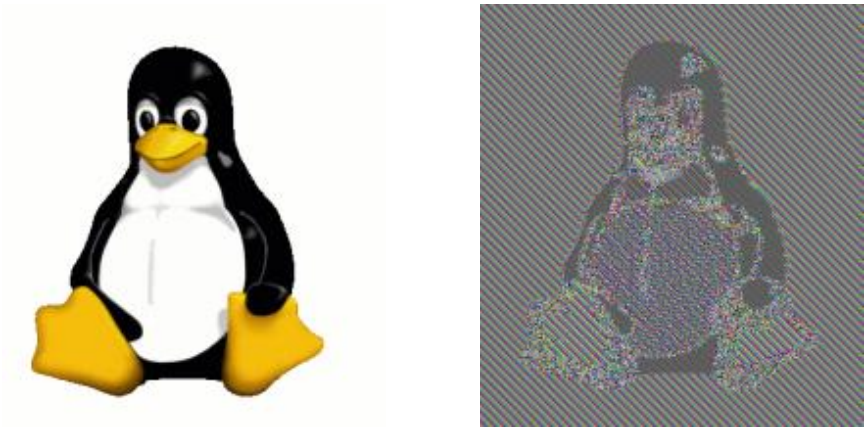

FIGURE 4 – Une illustration du problème de l'ECB

Dans l'illustration précédente, on chiffre chaque pixel indépendamment par la méthode ECB. La simple visualisation du fichier chiffré dévoile cependant énormément d'informations.

2.2 Enchaînement des blocs : « Cipher Block Chaining » (CBC)

Le CBC est une méthode consistant à utiliser le bloc précédemment chiffré pour effectuer une opération « ou exclusif » (XOR) sur le bloc suivant *non chiffré*. Après cette opération, on chiffre normalement le bloc obtenu à la suite de cette opération, et on continue ainsi sur les blocs suivants.

Pour le tout premier bloc, on utilise un *vecteur d'initialisation*. Il s'agit d'un vecteur de la taille d'un bloc (128 bits donc) que l'on utilise pour faire une opération OU exclusif sur le premier bloc. Ce vecteur peut être généré aléatoirement ou selon un procédé arbitraire, et peut être envoyé en clair sans compromettre la sécurité du chiffrement.

En fait, chiffrer le vecteur d'initialisation peux dans certains cas créer une faiblesse dans le chiffrement. Il est cependant important que ce vecteur soit le plus aléatoire possible, car un vecteur mal choisit peut également compromettre le chiffrement. Quoi qu'il en soit, il ne faut ***jamais*** réutiliser un vecteur d'initialisation pour une même clé, quelle que soit la méthode utilisée.

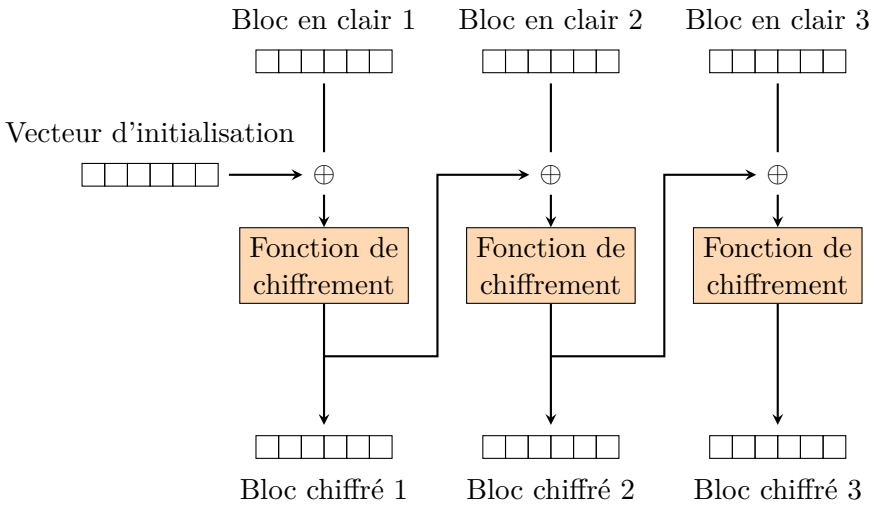

FIGURE 5 – Illustration du mode CBC.

2.3 Chiffrement basé sur un compteur : « CounTeR » (CTR)

Le mode CTR est basé sur un compteur. On prend un nombre aléatoire de la taille d'un bloc (un nonce), on le chiffre par AES, et on effectue l'opération OU EXCLUSIF sur le bloc du nombre chiffré avec le texte *en clair*. Pour chiffre le bloc suivant, on peut choisir un autre nombre aléatoirement, ou on peut tout simplement incrémenter le nombre précédent. Il a l'avantage de permettre un chiffrement par flux, au même rythme que les données arrivent. Cependant, si le nombre choisi venait à être corrompu, alors l'intégralité des données seraient perdues.

Le texte en clair n'est donc pas directement chiffré par la fonction de chiffrement, seule le nonce l'est.

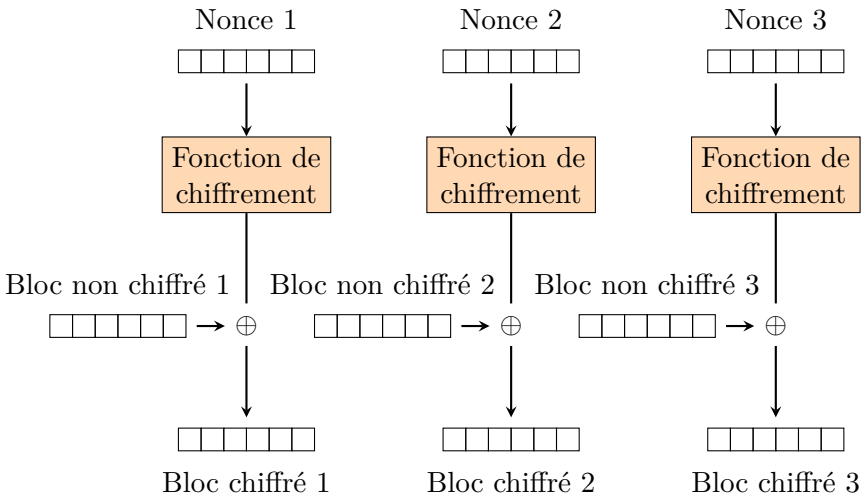

FIGURE 6 – Illustration du mode CTR.

3 Attaques

3.1 Attaque par manipulation de bits

L'attaque par « manipulation de bits » (bit-flipping attack) est une attaque contre le mode CBC et ne cible pas spécifiquement le chiffrement AES, mais tous les chiffrements par blocs utilisant le mode CBC. Elle ne permet pas de récupérer les données, mais de modifier son contenu de manière relativement contrôlée.

Le procédé est simple. Puisque dans le mode CBC, un bloc est est « xoré » avec le bloc précédent *avant* son propre chiffrement, alors modifier le bloc chiffré précédent change directement le contenu du bloc lors du déchiffrement.

Par exemple, soit $C_2[5]$ le cinquième octet du bloc 2 chiffré, et $B_3[5]$ le cinquième octet du bloc 3 non chiffré. Si on désire obtenir l'octet D à la place de $B_3[5]$, il faut que :

$$C_2[5] = B_3[5] \oplus D$$

Cependant, cette opération a pour effet de rendre le bloc modifié 2 complètement illisible, ses données quand on le déchiffre sont remplacées par un contenu aléatoires.

Ce problème ne survient pas quand on modifie le vecteur d'initialisation. Si il est modifié, alors seul le premier bloc est transformé, et cette modification n'a aucune influence sur le reste des blocs. Cette technique n'est pas nécessairement possible, car elle nécessite la connaissance du vecteur d'initialisation ainsi que la possibilité de l'envoyer à la fonction de déchiffrement.

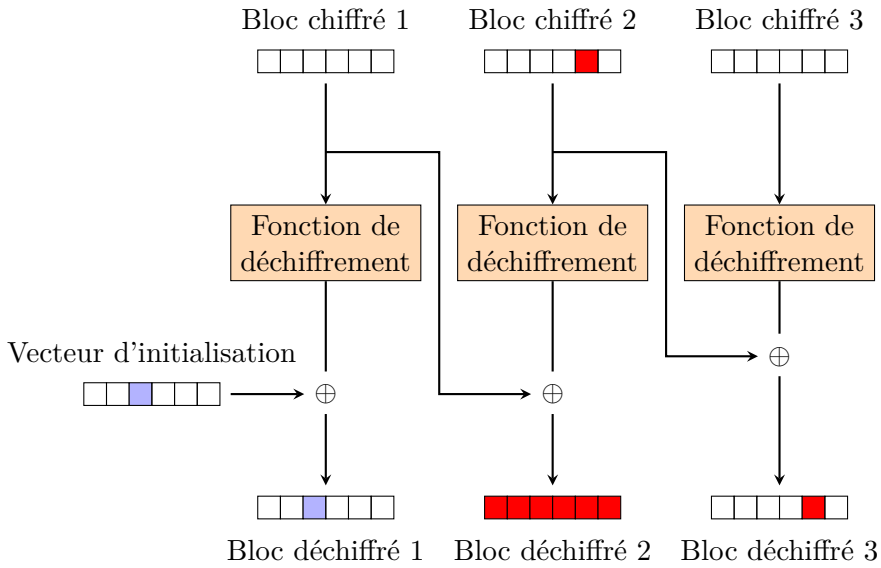

FIGURE 7 – Illustration de la manipulation.

Cette opération, même si elle ne permet pas de déchiffrer les données, a un intérêt particulier. Prenons une situation fictive où un virement bancaire est effectué, et que les instructions sont chiffrées par AES-CBC. Le contenu peut être :

« Virer l'argent du compte 4452 vers le 9851 ».

L'attaque par basculement de bit peut permettre de modifier ce message en

« Virer l'argent du compte 4452 vers le 0126 ».

De-même, si une communication entre un serveur et un client se fait par AES-CBC, il est possible de modifier des messages du type « authorized=0 » en « autorized=1 ».

En résumé, si vous connaissez le format du message, et que vous pouvez le modifier avant de l'envoyer, alors il est possible de contrôler le contenu qui sera lu par le receveur. Modifier le vecteur d'initialisation permet de contrôler le premier bloc sans contrepartie, mais il n'est pas toujours possible de choisir ce vecteur. Les autres blocs peuvent être modifiés à volonté, mais cette opération sacrifie un bloc qui sera complètement « détruit ».

3.2 Injection de blocs

Cette attaque vise le mode CBC. Le principe de l'injection de blocs est sensiblement le même que pour la manipulation de bit, au détail près qu'on ne modifie pas le bloc C_{n-1} pour modifier le bloc C_n, mais on injecte un nouveau bloc C' entre les deux, qui modifiera C_n. L'avantage de cette technique est qu'on ne sacrifie pas un bloc entier pour commettre l'attaque,

ce qui pourrait être problématique si l'intégrité de certaines données est importante.

3.3 Oracle de remplissage

Le chiffrement AES étant un chiffrement par blocs de 16 octets, il est nécessaire de faire du remplissage (*padding*) lorsque que le dernier bloc fait moins de 16 octets.

Le standard *PKCS#7*, le standard le plus utilisé, défini le remplissage de cette manière :

```
01
02 02
03 03 03
04 04 04 04
05 05 05 05 05
06 06 06 06 06 06
etc.
```

Ce standard stipule que s'il manque 4 octets pour faire un blocs, le remplissage sera quatre octets ayant chacun pour valeur 0x04. D'après la définition de la norme, il y a forcément au moins un octet de remplissage. Si il n'y en pas pas besoin, alors un bloc entier de remplissage est rajouté à la fin.

Wikipedia défini un oracle informatique ainsi : « procédure capable d'analyser le comportement observé du programme exécuté, et de décider si oui ou non une défaillance est présente ».

L'attaque de l'oracle de remplissage est une attaque où l'on connaît le dernier octet du dernier bloc, et qui permet de décrypter complètement celui-ci. Elle reprend en partie le fonctionnement de l'attaque par manipulation de bits.

3.3.1 Principe : ce qu'il faut comprendre

Il faut bien comprendre pour cette attaque que le fait que le message initial ait du remplissage ou non n'importe pas. Le principe de l'attaque est que l'on puisse savoir précisément quelle est la valeur d'un octet après déchiffrement suite à l'altération du bloc précédent. Le but est d'obtenir un bloc déchiffré contenant du remplissage connu. L'oracle va en fait révéler si le remplissage est correct, et par extension quelle est la valeur de celui-ci.

3.3.2 Explication détaillée

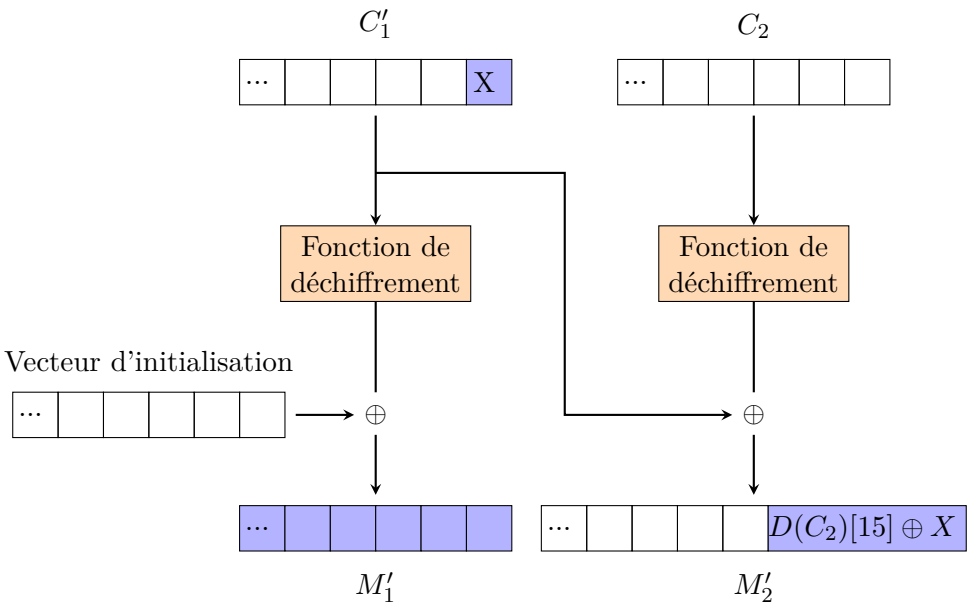

FIGURE 8 – Illustration de l'attaque

Soit un message constitué de deux blocs en clair (M_1, M_2), et soit D la fonction de déchiffrement. On considère les blocs de la forme $M_i = (M_i[0], M_i[1], ..., M_i[15])$.

On chiffre ces blocs en mode AES-CBC avec un vecteur d'initialisation C_{vi}, on obtient donc trois blocs (C_{vi}, C_1, C_2).

On remplace complètement tous les octets de C_1 par des valeurs aléatoires, ce qui nous donne C_1'. On envoie donc l'ensemble (C_{vi}, C_1', C_2) au serveur, qui nous dit alors si le remplissage obtenu lors du déchiffrement est correct. Si c'est incorrect, alors on réessaye avec un autre C_1', en incrémentant le dernier octet de C_1' qui est $C_1'[15]$.

S'il est correct, il faut encore déterminer quel est le remplissage. Il y a une forte probabilité que le remplissage en question soit 0x01, puisque par exemple si il vaut 0x02, alors il faut également que l'avant dernier octet vaille 0x02, ou alors il faudrait que les 3 derniers octets vaillent 0x03, et ainsi de suite pour des valeurs plus élevées.

Cependant, il faut quand même s'assurer d'avoir la bonne valeur. Pour cela, on incrémente $C_1'[14]$: si il n'y a pas d'erreur, alors ça veut dire que le dernier octet avait pour valeur 0x01. Sinon, on reprend les valeurs précédentes, et on incrémente $C_1'[13]$, et ainsi de suite jusqu'à qu'il n'y ait plus d'erreur. Le nombre d'octets que l'on ne peut pas modifier sans déclencher d'erreur ($C_1'[15]$ inclus) est alors le nombre d'octets servant de remplissage.

Supposons dans notre cas que modifier $C_1[14]$ déclenche une erreur. Cela veut dire que $D(C2)[15] \oplus C_1'[15] = 0x01$, soit $D(C_2)[15] = C_1[15]' \oplus 0x01$, donc

$$M_2[15] = C_1[15] \oplus C_1'[15] \oplus 0x01$$

On vient de récupérer le dernier octet de M_2 ! Le chiffrement est donc cassé.

On applique la même méthode aux autres octets en augmentant la taille du remplissage petit à petit. Puisqu'on connait la valeur $D(C_2)[15] + C_1'[15]$, on lui donne facilement la valeur 0x02 en manipulant C_1' par l'opération $C_1''[15] = 1 \oplus 2 \oplus C_1'[15]$, et on fait une attaque par force brute pour donner la valeur 0x02 à $M_2'[14]$. On sait qu'on a obtenu la bonne valeur quand l'oracle ne renvoie plus d'erreur. En répétant ce principe, on récupère toutes les données d'un bloc.

Si on avait plus de deux blocs, pour récupérer le deuxième bloc on applique le même principe, mais en omettant d'envoyer le troisième bloc à l'oracle.

Mais comment faire pour le premier bloc ? Tout dépend de la situation. Si on peut manipuler le vecteur d'initialisation, alors c'est parfait : il suffit d'appliquer la manipulation précédemment décrite au vecteur d'initialisation lui-même. Par contre, sans pourvoir disposer du vecteur, impossible de récupérer le premier bloc.

A Fonction de substitution

Les lignes indiquent les bits les plus significatifs, et les colonnes les moins significatifs. Par exemple, la substitution de *9c* est donc à la ligne *90* et à la colonne *0c*, donc est 0xDEh.

	00	**01**	**02**	**03**	**04**	**05**	**06**	**07**	**08**	**09**	**0a**	**0b**	**0c**	**0d**	**0e**	**0f**
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Table 2 – Table de substitution de Rijndael

	00	**01**	**02**	**03**	**04**	**05**	**06**	**07**	**08**	**09**	**0a**	**0b**	**0c**	**0d**	**0e**	**0f**
00	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
10	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
20	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
30	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
40	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
50	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
60	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
70	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
80	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
90	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a0	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b0	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c0	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d0	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e0	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f0	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Table 3 – Table de substitution inverse de Rijndael